Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 6

Course: Operating Systems

Topic: Inside the xv6 File System

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – READ WHAT XV6 SHOWS YOU **2**
1.1 Observe 2

2 PART 2 – WRITE MYSTAT **3**
2.1 Observe 4

3 PART 3 – MEASURE THE LIMITS **4**
3.1 Observe 5

4 PART 4 – WRITE FIND **5**
4.1 Stretch: the inode number with -n 7

Conclusions **7**

PURPOSE OF THE WORK

The purpose of this laboratory work is to look inside the file system of xv6, which has the same ideas as `ext4` (inodes, directories as tables of names, hard links, a log) in a few hundred lines of readable code. First the listing printed by `ls` is decoded with `kernel/stat.h`. Then the program `mystat` is written, which prints the type, the inode number, the number of links and the size of a file, and it is used to watch a hard link at work. Then the limits written in `kernel/fs.h` are measured (the length of a name, the largest file, the layout of the disk). Finally the program `find` is written, which walks the directory tree recursively. The work was done on the current xv6-riscv tree (commit `f780d78`), built with `riscv64-unknown-elf-gcc` and run in QEMU 8.2 on Ubuntu 24.04 (a Debian-family Linux); every output below was produced by these programs in xv6, on a fresh disk (`rm fs.img && make fs.img`, then `make qemu`). The answers to all the *Observe* boxes and to the *Stretch* task are given in the corresponding parts.

1 PART 1 – READ WHAT XV6 SHOWS YOU

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36792
echo           2 4 35640
forktest       2 5 17416
...
sync           2 22 35040
mystat         2 23 36336
find           2 24 38744
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

Figure 1.1 – ls on a fresh disk, and a new empty directory

The listing is shortened (the dots stand for the other programs). Each line is: name, type, inode number, size in bytes. The lines for `mystat` and `find` are there because the two programs of this work were added to the disk image.

1.1 Observe

1. **The types.** In `kernel/stat.h`: `T_DIR = 1` is a *directory*, `T_FILE = 2` is a regular *file*, and `T_DEVICE = 3` is a *device* (here `console`, type 3). There is no type for a symbolic link: xv6 knows only these three.

2. **The root directory.** Its inode number is **1**: the constant `ROOTINO` in `kernel/fs.h`, and the first line of Figure 1.1 shows `.` with inode 1. The entries `.` and `..` are the same here because the root has no parent: `mkfs` creates the root and writes both entries with the same inode number (`rootino`), so `..` of the root points to the root itself. (In the directory `a` the two entries differ: `.` is inode 26 and `..` is inode 1.) The size 1024 of the root is not the size of its entries: at the end `mkfs` rounds the size of the root directory up to a whole block (`BSIZE` = 1024 bytes).

3. **The size of an entry.** A directory entry is `struct dirent`: a 2-byte inode number (`ushort inum`) and a name of `DIRSIZ` = 14 bytes, that is **16 bytes**. The empty

directory `a` has size $32 = 2 \times 16$, so it contains two entries: `.` (itself, inode 26) and `..` (the parent, inode 1).

2 PART 2 – WRITE MYSTAT

The file `user/mystat.c` completes the template of the assignment: `typename` maps the type to a word, and `main` calls `stat` for every argument, writes an error to the descriptor 2 and goes to the next argument if it fails.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Figure 2.1 – user/mystat.c

The line `$U/_mystat\` (with a TAB at the start) was added to `UPROGS` in the `Makefile`, after `$U/_sync\` (Figure 2.2). The cast `(int)st.size` is needed because `size` is a 64-bit number and the `printf` of xv6 prints `%d` as an `int`.

```
UPROGS=\
    $U/_cat\
    ...
    $U/_sync\
    $U/_mystat\
    $U/_find\
```

Figure 2.2 – The new lines in UPROGS

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ mystat nofile
mystat: cannot stat nofile
```

Figure 2.3 – mystat on three kinds of file, and on a name that does not exist

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
```

```
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.4 – A hard link: after ln and after rm

2.1 Observe

1. **After `ln` and after `rm`.** Both outputs are in Figure 2.4. After `ln README r2` the two names show the *same* inode (2) and the link count of that inode is 2: there is one file with two names. After `rm README` the inode, the size (2441) and the data did not change; only the link count changed, from 2 to 1. The file is fully readable through the other name: `cat r2 | wc` gives 48 lines, 336 words, 2441 bytes, exactly the size of the original `README`. This is the same behaviour as `hard.txt` in Lab 5.

2. **Where the link count is decreased and when the blocks are freed.** In `kernel/sysfile.c`, the function `sys_unlink` first erases the entry from the directory (it writes a zeroed `struct dirent` with `writei`), and then at line **244** it executes `ip->nlink--;` followed by `iupdate(ip)` (the inode is written to the disk). (The other `nlink--`, at line 178, is the error path of `sys_link`.) `unlink` does not free any block itself. The data blocks are freed in `iput` (`kernel/fs.c`, line 365, the call `itrunc(ip)`), which is called by `iunlockput`, and only when the condition `ip->ref == 1 && ip->valid && ip->nlink == 0` is true: the link count is 0 *and* no other process or open file refers to the inode. Then `ifree` marks the inode free. So the blocks are released when the last name is removed and the last user has closed the file, the same rule that was seen with the deleted but open file in Lab 5.

3. **The fields of an xv6 inode.** `struct dinode` in `kernel/fs.h` has: `type` (file, directory or device), `major` and `minor` (only for devices), `nlink` (the number of links), `size` (in bytes) and `addrs[NDIRECT+1]` (the 12 direct block numbers and one number of an indirect block). That is $2+2+2+2+4+13\times 4 = 64$ bytes. There is no owner, no group, no permission bits, no times, and no type of symbolic link; this is why xv6 cannot have `chmod`, `ln -s` or "Permission denied". The name of a file is not in the inode but in the directory entry, as in Linux.

3 PART 3 – MEASURE THE LIMITS

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total
    2000
```

Figure 3.1 – The constants of kernel/fs.h and the layout of the disk

```
$ mkdir averyveryverylongname
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
...
console         3 25 0
averyveryveryl 1 26 32
$ mystat averyveryverylongname
```

```
averyveryverylongname: directory, inode 26, links 1, size 32 bytes
```

Figure 3.2 – A name longer than DIRSIZ

3.1 Observe

1. **The long name.** `ls` shows `averyveryveryl`: only the first **14** characters of the 21-character name. The constant is `DIRSIZ = 14`, the size of the field `name` in `struct dirent`; a longer name is cut when the entry is written. The directory was created (type 1, inode 26, size 32), and `mystat` with the full name finds it, because the lookup compares only the first `DIRSIZ` characters too. A consequence is that two different names with the same first 14 characters are the same name for xv6.

2. **The largest file.** The inode has 12 direct blocks and one indirect block. The indirect block (1024 bytes) holds numbers of 4 bytes each, so `NINDIRECT` $= 1024/4 = 256$ numbers. The maximum number of blocks is `MAXFILE` $= 12 + 256 = 268$, and

   $$268 \times 1024 = 12 \times 1024 + 256 \times 1024 = 12\,288 + 262\,144 = 274\,432 \text{ bytes} = 268 \text{ KiB}.$$

   (This small limit is why a big file cannot be created in xv6; file systems such as ext2 and ext3 add a double and a triple indirect block to the inode to allow bigger files, and ext4 uses extents.)

3. **The layout of the disk.** From the line `nmeta 47 ...blocks 1953 total 2000` (Figure 3.1): the whole disk has **2000 blocks** (`FSSIZE` in `kernel/param.h`), of 1024 bytes, about 2 MB. The meta blocks are `nmeta` $= 47$: 1 boot block, 1 super block, **31** blocks of the log (30 data blocks `LOGBLOCKS` + 1 header), **13** blocks of inodes ($200/16 + 1$: 200 inodes, 16 inodes of 64 bytes in a block, plus one) and **1** block of the free-block bitmap. Check: $1 + 1 + 31 + 13 + 1 = 47$. The rest, $2000 - 47 =$ **1953** blocks, are the data blocks.

4 PART 4 – WRITE FIND

`find <dir> <name>` prints every path under `<dir>` whose last component is `<name>`. The program (Figure 4.1) is built on `user/ls.c`: it opens the path, calls `fstat`, and if the path is a directory it reads the `struct dirent` entries one by one, builds the full path of each entry in a buffer and calls itself on it. The function `basename` returns the part of the path after the last slash, and `strcmp` compares it with the wanted name. The entries with `inum == 0` (free slots) and the entries `.` and `..` are skipped, and every opened descriptor is closed on every way out of the function, because a process can have only 16 open files (`NOFILE`). The third argument `-n` (Stretch) is described in the last subsection.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

int shownum = 0; // set by the optional third argument "-n"

// Return the last component of path.
char *
basename(char *path)
{
  char *p;
```

```
  for(p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  return p + 1;
}

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(strcmp(basename(path), name) == 0){
    if(shownum)
      printf("%s %d\n", path, st.ino);
    else
      printf("%s\n", path);
  }

  if(st.type == T_DIR){
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
      fprintf(2, "find: path too long\n");
      close(fd);
      return;
    }
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';
    while(read(fd, &de, sizeof(de)) == sizeof(de)){
      if(de.inum == 0)
        continue;
      if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
        continue;
      memmove(p, de.name, DIRSIZ);
      p[DIRSIZ] = 0;
      find(buf, name);
    }
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc != 3 && !(argc == 4 && strcmp(argv[3], "-n") == 0)){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  if(argc == 4)
    shownum = 1;
  find(argv[1], argv[2]);
  exit(0);
}
```

Figure 4.1 – user/find.c

The path is compared *before* the directory is read, so the starting path itself can match (this gives the line `a/b` in the second test of Figure 4.2). The names of `struct dirent` are not always ended by a zero (a name of 14 characters fills the field), so the program copies the name with `memmove` and puts the zero itself at `p[DIRSIZ]`, exactly as `ls` does; the comparison `strcmp(de.name, ".")` before that copy works for the names `.` and `..` because they are shorter than the field and are ended by zeros. The test of the assignment was run on a fresh

disk:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find
usage: find <dir> <name> [-n]
$ find nodir x
find: cannot open nodir
```

Figure 4.2 – The test of find on a fresh disk

The three files named `r2` were found, in the same order as in the assignment, `find a b` printed `a/b`, and the two error cases (no arguments, a directory that does not exist) print a message on the standard error and do not crash.

4.1 Stretch: the inode number with -n

With the third argument `-n`, the program prints the inode number after each path (`st.ino` of the `fstat` that is already done, so no extra system call is needed).

```
$ find . r2 -n
./a/b/r2 28
./a/r2 29
./r2 3
$ find . cat -n
./cat 3
```

Figure 4.3 – find with -n: which r2 is the same file as cat?

Only `./r2` is the same file as `cat`: it has the inode number **3**, the same as `./cat`, because `ln cat r2` made a second name (a hard link) for that inode; the same `find . cat -n` shows only one path for inode 3 because the search is by name, not by inode. The two other files have their own inodes, 28 (`a/b/r2`) and 29 (`a/r2`), created by `echo`; they only have the same *name* as the link, which is the point of the exercise: a name and an inode are different things.

CONCLUSIONS

In this laboratory work the file system of xv6 was studied from the user level and from the code. It was seen that the listing of `ls` is a view of `struct stat` and of the directory entries: a directory is a file of 16-byte entries (a 2-byte inode number and a 14-byte name), so an empty directory has size 32 and the root has inode 1, with `.` and `..` pointing to itself. The program `mystat` was written and it showed with real output that a hard link is a second name for the same inode, that `rm` decreases the link count (line 244 of `sys_unlink`) and that the blocks are freed by `iput` only when the count is 0 and nobody has the file open. An xv6 inode (64 bytes) has no owner, permissions, times or symbolic links.

The measurements of `kernel/fs.h` gave the limits of this small system: names are cut at 14 characters, the largest file is $(12 + 256) \times 1024 = 274\,432$ bytes, and the disk of 2000 blocks has 47 meta blocks (1 boot, 1 super, 31 log, 13 inode, 1 bitmap) and 1953 data blocks. The program `find` walks the tree recursively with the same scheme as `ls`, skipping free slots, `.` and `..`, and closing every descriptor; with the option `-n` it confirmed that only

`./r2` is the same file as `cat`. In this way the ideas of Lab 5 (inode, name, link count) were checked in the code of a real, small file system.